EXAFS Quick Reference Guide

Assumes Python environment with `EXAFSAll`*, LARCH, and helper utilities*

---

0) Session Setup, LARCH Parameters, and I/O Utilities

Session Setup

```
# Initialize EXAFS analysis session
exafs = EXAFSAll()

# Optional: project-aware working directory
base_dir = project_number("########")   # e.g., "2026030152" or Path to Directory//
# base_dir = "/Path_to_Dir/"
```

LARCH Processing Parameters

```
LARCH_PARAMS = {
    "pre_edge": dict(
        pre1=-150,
        pre2=-30,
        nvict=1,
        npre=1,
        nnorm=1,
        norm1=1,
        norm2=-1
    ),

    "autobk": dict(
        rbkg=1.0,     # Critical change
        kmin=0,       # Suppress low-k noise
        kmax=7.5,
        kweight=2,
        clamp_lo=1.0,
        clamp_hi=20.0
    ),

    # k  R Fourier transform
    "xftf": dict(
        kmin=0,
        kmax=7.5,
        kweight=2,
        dk=0.02,
        window="hanning",
        rmax_out=4
    ),

    # R  k inverse transform (shell filtering)
    "xftr": dict(
        rmin=1.0,
        rmax=4,
        kweight=2,
        dr=0.25,
```

```
        window="Kaiser-Bessel"
    )
}
```

Saving and Exporting Processed Signals

```
# Save SUM or single-scan normalized data using display labels
exafs.save_signals(folder="Paths", y="flat",
                   summed_only=True, use_display_label=True)

exafs.save_signals(folder="Paths", y="norm",
                   summed_only=True, use_display_label=True)

# Export SUM or per-group XDI .dat files (custom labels included)
exafs.export_groups_xdi_dat(outdir="/Paths/")
```

Conventions and Common Parameters

- **Labels:** Use a single string (e.g., `"Scan1"`) or a list (e.g., `["Scan1","Scan2"]`).
- **Groups:** SUM groups can be expanded using `show_members=True`.
- **Transmission:** `trans_only=True` plots transmission only.
- **Stacking:** Use `offset_step` or `stack="subplots"`.
- **Ranges:** Use `kmin/kmax` and `rmin/rmax` to restrict data.
- **k-weight:** Choose 1, 2, or 3 depending on analysis.

---

1) $\mu(E)$ Energy Scans `exafs.plot_mu()`

Basic Usage

```
exafs.plot_mu()
exafs.plot_mu("Scan1")
exafs.plot_mu(["Scan1","Scan2","Scan3"])
exafs.plot_mu(samples)   # where samples is a SUM group
```

Display Modes

```
exafs.plot_mu("Scan1", show_raw=True)
exafs.plot_mu("Scan1", show_norm=True)
exafs.plot_mu("Scan1", show_flat=True)

# Hide diagnostic overlays
exafs.plot_mu("Scan1", show_pre_edge=False, show_post_edge=False,
              show_bkg=False, show_bkgsub=False)
```

Transmission-Related

```
exafs.plot_mu("Scan1", show_trans=True, show_norm=True)
exafs.plot_mu("Scan1", trans_only=True)
exafs.plot_mu(samples, trans_only=True)
```

Members and Stacking

```
exafs.plot_mu(samples, show_members=True)
exafs.plot_mu(samples, show_members=True, members_only=True)
exafs.plot_mu(["A","B","C"], offset_step=0.4)
```

Axes

```
exafs.plot_mu("Scan1", xlim=(7100,7400), ylim=(0,3), title="Fe K-edge")
```

---

2) $\chi(k)$ `exafs.plot_chi_k()`

Basic Usage

```
exafs.plot_chi_k("Scan1")
exafs.plot_chi_k(["Scan1","Scan2"])
exafs.plot_chi_k(samples)
```

k-Range and Weighting

```
exafs.plot_chi_k("Scan1", kmin=2, kmax=12)
exafs.plot_chi_k("Scan1", kweight=2)
```

Group Visualizations

```
exafs.plot_chi_k(samples, show_members=True)
exafs.plot_chi_k(samples, show_members=True, members_only=True)

# Per-member subplots
exafs.plot_chi_k(samples, show_members=True,
                 stack="subplots", sharey=True)
```

---

3) $\chi(R)$ `exafs.plot_chi_r()`

Basic Usage

```
exafs.plot_chi_r("Scan1")
exafs.plot_chi_r(samples)
```

Components

```
exafs.plot_chi_r("Scan1", show_mag=True)
exafs.plot_chi_r("Scan1", show_real=True, show_imag=True)
exafs.plot_chi_r("Scan1", kweight=2)
```

Ranges and Members

```
exafs.plot_chi_r("Scan1", rmin=0, rmax=5)
exafs.plot_chi_r(samples, show_members=True)
exafs.plot_chi_r(samples, show_members=True, members_only=True)
```

Subplots

```
exafs.plot_chi_r(samples, show_members=True,
                 stack="subplots", sharey=True)
```

---

4) Wavelet Transform (WT) `exafs.plot_wavelet()`

Basic Usage

```
exafs.plot_wavelet("Scan1")
exafs.plot_wavelet(["Scan1","Scan2"])
exafs.plot_wavelet(samples)
```

Range Selection and Colormap

```
exafs.plot_wavelet("Scan1", kmin=2, kmax=12)
exafs.plot_wavelet("Scan1", rmin=0, rmax=6)
exafs.plot_wavelet("Scan1", cmap="viridis")
```

Members and Multi-Panel Views

```
exafs.plot_wavelet(samples, show_members=True)
exafs.plot_wavelet(samples, show_members=True,
                   stack=True, share_clim=True)
exafs.plot_wavelet(samples, show_members=True,
                   members_only=True, stack=False)
```

---

5) Workflows and Quick Checks

```
# Compare PFY vs transmission
exafs.plot_mu("Scan1", show_raw=True, show_trans=True)

# Alignment sanity check
exafs.plot_mu("Scan1")
exafs.plot_chi_k("Scan1")
exafs.plot_chi_r("Scan1")

# Compare SUM group vs individuals
exafs.plot_mu(samples, show_members=True)
exafs.plot_chi_k(samples, show_members=True)
exafs.plot_chi_r(samples, show_members=True)
```

---

6) Single-File Reduction and Analysis `exafs.align_multiple()`

```
## Single file data reduction and analysis
exafs = EXAFSAll()
filename = 'file.dat'
full_path = os.path.join(base_dir, filename)
aligned_data, e0 = exafs.align_multiple(larch_params=LARCH_PARAMS,
    files= {"Scan1": full_path},# dict like {"Scan1": "/path/to/file1", ...}
    inb_filter="AsKa1_spectra",  # Excitation edge starting with the element name
    reference=None,
    summed_groups=None, # None | summed_groups | All
    ion_chamber="I0",
    channel_side="OutB",  # inB or OutB channel for analysis
    auto_rbkg=False,   # default; can omit
    min_snr_post = 21.0,      # edge_jump / rms_post_residual
    plot_mode="aligned_mu",            # returns (aligned_data, e0)
    prompt_save=False,   # no export here
    deglitch_mode='none', #'pfy',  # 'pfy'  | 'both' | 'none'.
    deglitch_region="exafs",  # xanes, exafs or custom
    deglitch_xrange=(8330.0, 8390.0),  # tuple OR dict of named ranges
)

# Optional print using safe labels
# If your dataset exposes 32 InB/OutB channels matching inb_filter,
# this makes a 66 grid (36 slots), with the 33rd for sum(good)
exafs.plot_good_channels('Scan1', ncols=6, normalize_each=True)
# per-label metrics DataFrame
exafs.metrics["Scan1"].head(34)
# Optional print using safe labels
exafs.print_e0_values()
# Plot pre-computed processed signals
#plot mu for single data and the other reduction data
exafs.plot_mu(
        xlim=(11600,12800),
        show_raw = True,
        show_pre_edge = True,
        show_post_edge = True,
        show_bkg = True,
        show_bkgsub = False,
        show_norm = False,
        show_flat = False,
        offset_step = 0.0)
#plot mu for single flat or norm data
exafs.plot_mu(
        xlim=(11600,12800),
        show_raw = False,
        show_pre_edge = False,
        show_post_edge = False,
        show_bkg = False,
        show_bkgsub = False,
        show_norm = True,
        show_flat = True,
)
```

```
exafs.plot_chi_k("Scan1")
exafs.plot_chi_r("Scan1")
exafs.plot_wavelet("Scan1")
```

---

7) Multi-File Reduction and Analysis

prepare_inputs_any() & exafs.align_multiple()

```
exafs = EXAFSAll()
# base_dir = project_number("########")
base_dir = "Path_to_Dir/"
# --- Run the preparation ---
files, filenames_sorted, groups_by_token, label_map, summed_groups = prepare_inputs_any(
    base_dir=base_dir,
    # exclude_glob = ['*'],   # Exclude data files you want to plot
    include_glob = ['EL_rbc_AsSe_AsK_1_EXAFS_*'],  # Include data files you want to plot
    scan_label_prefix=None,  # kept in signature; labels ignore prefixes by design now
    prefer_energy_first=True, # energy groups listed first (if any)
    reset_scan_numbers_per_group=False,# GLOBAL indices (recommended for EXAFS workflows)
    preview_input=True,
)

# Align and Sum (Multi-file)
aligned_data, e0 = exafs.align_multiple(larch_params=LARCH_PARAMS,
    files=files,                        # dict like {"Scan1": "/path/to/file1", ...}
    inb_filter="AsKa1",   # Excitation edge starting with the element name
    ion_chamber="I0",     # ion chamber for noralization
    summed_groups=summed_groups,    # None for single files
    plot_mode="As_k-sums_only",            # returns (aligned_data, e0)
    channel_side="OutB",
    auto_rbkg=False,   # default; can omit
    min_snr_post = 21.0,      # edge_jump / rms_post_residual
    prompt_save=False,   # no export here
    deglitch_mode='none', #'pfy',  # 'pfy'  | 'both' | 'none'
    deglitch_region="custom",  # 'xanes'| 'exafs' | 'custom'
    deglitch_xrange=(12000.0, 12700.0),  # tuple OR dict of named ranges
)


# Lit the sample group name
samples = sample_labels(label_map)

# Optional print using safe labels
exafs.print_e0_values()

exafs_Ni.print_e0_values()

# Follow-up plots
# Plot (Multi-file)
```

```
exafs.plot_mu(
        # xlim=(6400,7100),
        show_raw = False,
        show_pre_edge = False,
        show_post_edge = False,
        show_bkg = False,
        show_bkgsub = False,
        show_norm = False,
        show_flat = True,
        offset_step = 1.0)
# Plotting k-space for group data
exafs.plot_chi_k(labels=samples, show_raw = False,stack="subplots", sharey=False)
#, outdir="/Paths/ND")
# Plotting k-space for group data
exafs.plot_chi_r(labels=samples, stack="subplots", sharey=False)
#, outdir="/Paths/ND")
# Plotting wavelet-space for group data
exafs.plot_wavelet(label=samples, stack=True)
#, outdir="/Paths/ND")
```

Notes: Examples are representative and flexible; combine flags as required. Transmission overlays require the corresponding transmission channels to be present. Ensure the environment defines `LARCH_PARAMS`, and helper utilities such as `project_number` and `prepare_inputs_any`.